AI ASSISTENT CODING 12.3

Name : Baisa Vaishanvi
2303A52142
Batch - 41

Task 1: Sorting Student Records for Placement Drive
Scenario
SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by CGPA in descending order.
Tasks
1. Use GitHub Copilot to generate a program that stores student records (Name, Roll Number, CGPA).
2. Implement the following sorting algorithms using AI assistance:
o Quick Sort
o Merge Sort
3. Measure and compare runtime performance for large datasets.
4. Write a function to display the top 10 students based on CGPA.

Prompt:
Generate a Sorting Student Records for Placement Drive
Generate a program (preferably in Python or Java) to manage student records containing Name, Roll Number, and CGPA.Use GitHub Copilot / AI assistance to implement Quick Sort and Merge Sort to sort students by CGPA in descending order.
Create a large dataset of student records and measure runtime performance of both algorithms using appropriate timing functions.Compare and print the execution time of Quick Sort vs Merge Sort.
Finally, implement a function to display the top 10 students based on CGPA with clear, formatted output.

Code:

C: > Users > Hello > 12.1.py > ...

```python
#Generate a Sorting Student Records for Placement Drive
#Generate a program (preferably in Python or Java) to manage student records containing Name, Roll Numbe
#Create a large dataset of student records and measure runtime performance of both algorithms using appr
#Finally, implement a function to display the top 10 students based on CGPA with clear, formatted output
import random
import time
class Student:
    def __init__(self, name, roll_number, cgpa):
        self.name = name
        self.roll_number = roll_number
        self.cgpa = cgpa

    def __repr__(self):
        return f"{self.name} (Roll: {self.roll_number}, CGPA: {self.cgpa})"
def generate_students(num_students):
    students = []
    for i in range(num_students):
        name = f"Student_{i+1}"
        roll_number = i + 1
        cgpa = round(random.uniform(0, 10), 2)
        students.append(Student(name, roll_number, cgpa))
    return students
def quick_sort(students):
    if len(students) <= 1:
        return students
    pivot = students[len(students) // 2].cgpa
    left = [x for x in students if x.cgpa > pivot]
    middle = [x for x in students if x.cgpa == pivot]
    right = [x for x in students if x.cgpa < pivot]
```

```python
def quick_sort(students):
    return quick_sort(left) + middle + quick_sort(right)
def merge_sort(students):
    if len(students) <= 1:
        return students
    mid = len(students) // 2
    left_half = merge_sort(students[:mid])
    right_half = merge_sort(students[mid:])
    return merge(left_half, right_half)
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i].cgpa > right[j].cgpa:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
def display_top_students(students, top_n=10):
    print(f"Top {top_n} Students based on CGPA:")
    for student in students[:top_n]:
        print(student)
if __name__ == "__main__":
    num_students = 1000
    students = generate_students(num_students)
```

```
58
59      start_time = time.time()
60      sorted_students_quick = quick_sort(students)
61      end_time = time.time()
62      print(f"Quick Sort Time: {end_time - start_time:.4f} seconds")
63
64      start_time = time.time()
65      sorted_students_merge = merge_sort(students)
66      end_time = time.time()
67      print(f"Merge Sort Time: {end_time - start_time:.4f} seconds")
68
69      display_top_students(sorted_students_quick)
70      display_top_students(sorted_students_merge)
```

Output :

```
PROBLEMS 1  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                    Python

PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Hello\AppData\Local\Python\pytho
hon.exe c:/Users/Hello/12.1.py
Quick Sort Time: 0.0020 seconds
Merge Sort Time: 0.0032 seconds
Top 10 Students based on CGPA:
Student_659 (Roll: 659, CGPA: 10.0)
Student_276 (Roll: 276, CGPA: 9.99)
Student_290 (Roll: 290, CGPA: 9.99)
Student_313 (Roll: 313, CGPA: 9.99)
Student_707 (Roll: 707, CGPA: 9.99)
Student_190 (Roll: 190, CGPA: 9.98)
Student_472 (Roll: 472, CGPA: 9.98)
Student_708 (Roll: 708, CGPA: 9.98)
Student_809 (Roll: 809, CGPA: 9.97)
Student_913 (Roll: 913, CGPA: 9.97)
Top 10 Students based on CGPA:
Student_659 (Roll: 659, CGPA: 10.0)
Student_707 (Roll: 707, CGPA: 9.99)
Student_313 (Roll: 313, CGPA: 9.99)
Student_290 (Roll: 290, CGPA: 9.99)
Student_276 (Roll: 276, CGPA: 9.99)
Student_708 (Roll: 708, CGPA: 9.98)
Student_472 (Roll: 472, CGPA: 9.98)
Student_190 (Roll: 190, CGPA: 9.98)
Student_913 (Roll: 913, CGPA: 9.97)
Student_809 (Roll: 809, CGPA: 9.97)
PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code>
```

Justification :

This task uses AI tools to efficiently store and sort student records based on CGPA. Sorting in descending order helps the Training and Placement Cell quickly identify top-performing students. Implementing Quick Sort and Merge Sort allows comparison between two efficient algorithms, ensuring correct ranking and understanding of performance differences on large datasets.

Task 2: Implementing Bubble Sort with AI Comments

• Task: Write a Python implementation of Bubble Sort.
• Instructions:
• Students implement Bubble Sort normally.

• Ask AI to generate inline comments explaining key logic (like swapping, passes, and termination).
• Request AI to provide time complexity analysis

Prompt:
Write a Python program to implement the Bubble Sort algorithm for sorting a list of numbers in ascending order.
Add AI-generated inline comments explaining the key logic, including comparisons, swapping of elements, number of passes, and when the algorithm stops.
Ensure the code is clean, readable, and beginner-friendly.
After the code, provide a time complexity analysis of Bubble Sort for best case, average case, and worst case in simple terms.

Code:

C: > Users > Hello > 12.2.py > ...

```python
#Write a Python program to implement the Bubble Sort algorithm for sorting a list of numbers in ascending order.
#Add AI-generated inline comments explaining the key logic, including comparisons, swapping of elements, number of pa
#Ensure the code is clean, readable, and beginner-friendly.After the code, provide a time complexity analysis of Bubb

def bubble_sort(numbers):
    n = len(numbers)
    # Outer loop for number of passes
    for i in range(n):
        swapped = False  # Flag to check if any swapping occurred in this pass
        # Inner loop for comparisons in each pass
        for j in range(0, n - i - 1):
            # Compare adjacent elements
            if numbers[j] > numbers[j + 1]:
                # Swap elements if they are in wrong order
                numbers[j], numbers[j + 1] = numbers[j + 1], numbers[j]
                swapped = True
        # If no swapping occurred, the list is already sorted
        if not swapped:
            break

# Example usage:
numbers = [64, 34, 25, 12, 22, 11, 90]
print("Original list:", numbers)
bubble_sort(numbers)
print("Sorted list:", numbers)

# Time Complexity Analysis:
# Best Case: O(n) - when the list is already sorted (only one pass is needed)
# Average Case: O(n^2) - when elements are randomly ordered
# Worst Case: O(n^2) - when the list is sorted in reverse order (maximum number of comparisons and swaps)
```

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                                   Python

PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Hello\AppData\Local\Python\pytho
e c:/Users/Hello/12.2.py
Original list: [64, 34, 25, 12, 22, 11, 90]
Sorted list: [11, 12, 22, 25, 34, 64, 90]
PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code>
```

Justification :
Bubble Sort is chosen because it is simple and easy to understand for beginners. AI-generated inline comments help explain the logic behind comparisons, swapping, and loop termination. Time complexity analysis improves understanding of why Bubble Sort is inefficient for large datasets, making this task useful for learning algorithm fundamentals.

Task 3: Quick Sort and Merge Sort Comparison
• Task: Implement Quick Sort and Merge Sort using recursion.
• Instructions:
• Provide AI with partially completed functions for recursion.
• Ask AI to complete the missing logic and add docstrings.
• Compare both algorithms on random, sorted, and reverse-sorted lists.
• Expected Output:
• Working Quick Sort and Merge Sort implementations.
• AI-generated explanation of average, best, and worst-case complexities.

Prompt:
Given partially completed recursive functions for Quick Sort and Merge Sort in Python, complete the missing logic using proper recursion.
Add clear docstrings explaining the purpose, parameters, and return values of each function.
Generate test cases using random, already sorted, and reverse-sorted lists and apply both algorithms to them.
Measure and compare execution time for each input type.
Finally, provide an AI-generated explanation of best-case, average-case, and worst-case time complexities for Quick Sort and Merge Sort in simple terms.

Code:

C: > Users > Hello > 12.3.py.py > ...

```python
#Given partially completed recursive functions for Quick Sort and Merge Sort in Python, complete the missing log
#Generate test cases using random, already sorted, and reverse-sorted lists and apply both algorithms to them.Me
#Finally, provide an AI-generated explanation of best-case, average-case, and worst-case time complexities for Q
import random
import time
def quick_sort(arr):
    """
    Sorts an array using the Quick Sort algorithm.

    Parameters:
    arr (list): The list of elements to be sorted.

    Returns:
    list: A new sorted list.
    """
    if len(arr) <= 1:
        return arr
    else:
        pivot = arr[len(arr) // 2]
        left = [x for x in arr if x < pivot]
        middle = [x for x in arr if x == pivot]
        right = [x for x in arr if x > pivot]
        return quick_sort(left) + middle + quick_sort(right)
def merge_sort(arr):
    """
    Sorts an array using the Merge Sort algorithm.

    Parameters:
    arr (list): The list of elements to be sorted.
```

```python
def merge_sort(arr):
    Returns:
    list: A new sorted list.
    """
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])

    return merge(left_half, right_half)
def merge(left, right):
    """Merges two sorted lists into a single sorted list.
    Parameters:
    left (list): The first sorted list.
    right (list): The second sorted list.
    Returns:
    list: A merged and sorted list containing all elements from both input lists.
    """
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
```

```
41  def merge(left, right):
58      result.extend(left[i:])
59      result.extend(right[j:])
60      return result
61  # Test cases
62  test_cases = {
63      "random": [random.randint(1, 100) for _ in range(10)],
64      "sorted": list(range(10)),
65      "reverse_sorted": list(range(10, 0, -1))
66  }
67  for key, arr in test_cases.items():
68      print(f"Testing {key} list: {arr}")
69
70      start_time = time.time()
71      sorted_arr_quick = quick_sort(arr)
72      end_time = time.time()
73      print(f"Quick Sort Result: {sorted_arr_quick}, Time taken: {end_time - start_time:.6f} seconds")
74
75      start_time = time.time()
76      sorted_arr_merge = merge_sort(arr)
77      end_time = time.time()
78      print(f"Merge Sort Result: {sorted_arr_merge}, Time taken: {end_time - start_time:.6f} seconds")
79      print("-" * 50)
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Hello\AppData\Local\Python\pythoncore
lo/12.3.py.py
Testing random list: [60, 70, 48, 36, 79, 99, 86, 97, 30, 82]
Quick Sort Result: [30, 36, 48, 60, 70, 79, 82, 86, 97, 99], Time taken: 0.000022 seconds
Merge Sort Result: [30, 36, 48, 60, 70, 79, 82, 86, 97, 99], Time taken: 0.000029 seconds
--------------------------------------------------
Testing sorted list: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
Quick Sort Result: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9], Time taken: 0.000014 seconds
Merge Sort Result: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9], Time taken: 0.000015 seconds
--------------------------------------------------
Testing reverse_sorted list: [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
Quick Sort Result: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], Time taken: 0.000009 seconds
Merge Sort Result: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], Time taken: 0.000013 seconds
--------------------------------------------------
PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code>
```

Justification:

This task demonstrates the use of recursion in sorting algorithms. By comparing Quick Sort and Merge Sort on random, sorted, and reverse-sorted data, students understand how input order affects performance. AI explanations of best, average, and worst-case complexities help in analyzing algorithm efficiency in real-world scenarios.

Task 4 (Real-Time Application – Inventory Management System)

Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.

Task:
• Use AI to suggest the most efficient search and sort algorithms for this use case.
• Implement the recommended algorithms in Python.
• Justify the choice based on dataset size, update frequency, and performance requirements.

Prompt:
Design a real-time Inventory Management System in Python for a retail store containing thousands of products with attributes Product ID, Name, Price, and Stock Quantity.
Use AI assistance to recommend the most efficient searching and sorting algorithms for this use case.
Create a table mapping each operation (search/sort) to the chosen algorithm with justification based on dataset size, update frequency, and performance needs.
Implement Python functions to search products by ID or name and sort products by price or quantity.
Ensure the output is clear, efficient, and suitable for a real-world application.

Code:

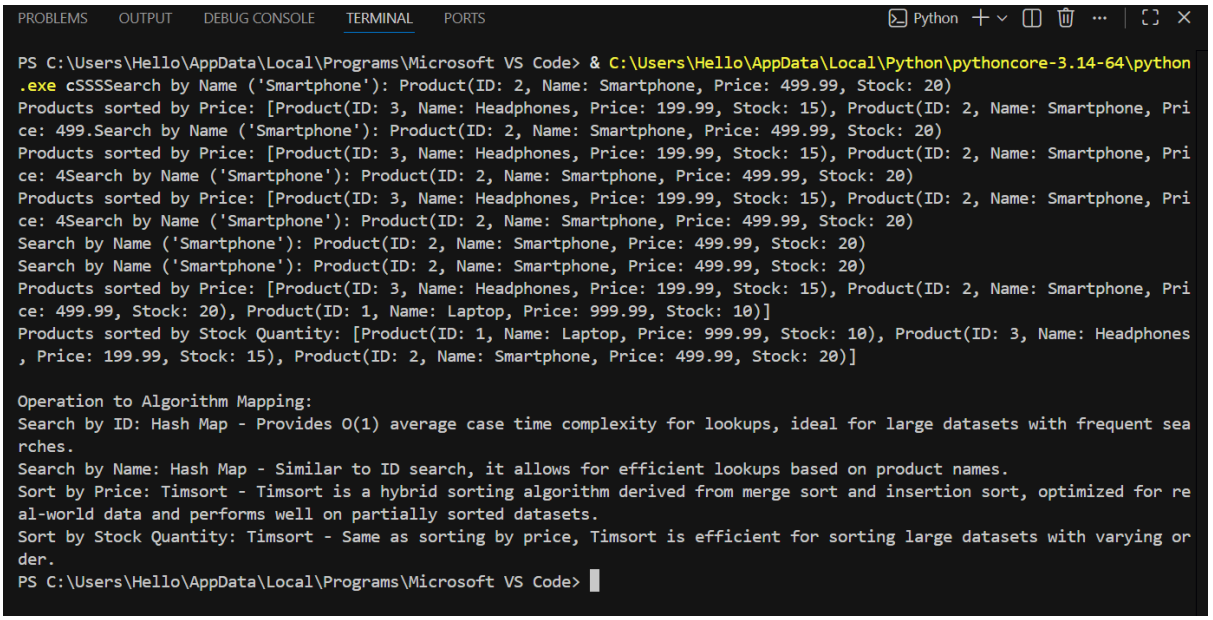
C: > Users > Hello > 12.4.py > ...

```python
#Design a real-time Inventory Management System in Python for a retail store containing thousands of p
#Use AI assistance to recommend the most efficient searching and sorting algorithms for this use case.
#Create a table mapping each operation (search/sort) to the chosen algorithm with justification based
#Implement Python functions to search products by ID or name and sort products by price or quantity.Er
import bisect
class Product:
    def __init__(self, product_id, name, price, stock_quantity):
        self.product_id = product_id
        self.name = name
        self.price = price
        self.stock_quantity = stock_quantity

    def __repr__(self):
        return f"Product(ID: {self.product_id}, Name: {self.name}, Price: {self.price}, Stock: {self.s
class InventoryManagementSystem:
    def __init__(self):
        self.products = []
        self.product_id_index = {}
        self.product_name_index = {}

    def add_product(self, product):
        # Add product to the inventory
        self.products.append(product)
        # Update indices for efficient searching
        self.product_id_index[product.product_id] = product
        self.product_name_index[product.name] = product

    def search_by_id(self, product_id):
        # Search for a product by ID using a hash map (O(1) average case)
```

```
class InventoryManagementSystem:
    def search_by_id(self, product_id):
        return self.product_id_index.get(product_id, None)

    def search_by_name(self, name):
        # Search for a product by name using a hash map (O(1) average case)
        return self.product_name_index.get(name, None)

    def sort_by_price(self):
        # Sort products by price using Timsort (O(n log n))
        return sorted(self.products, key=lambda x: x.price)

    def sort_by_stock_quantity(self):
        # Sort products by stock quantity using Timsort (O(n log n))
        return sorted(self.products, key=lambda x: x.stock_quantity)
# Example usage
if __name__ == "__main__":
    inventory = InventoryManagementSystem()
    inventory.add_product(Product(1, "Laptop", 999.99, 10))
    inventory.add_product(Product(2, "Smartphone", 499.99, 20))
    inventory.add_product(Product(3, "Headphones", 199.99, 15))

    print("Search by ID (1):", inventory.search_by_id(1))
    print("Search by Name ('Smartphone'):", inventory.search_by_name("Smartphone"))
    print("Products sorted by Price:", inventory.sort_by_price())
    print("Products sorted by Stock Quantity:", inventory.sort_by_stock_quantity())
```

```
# Table mapping operations to algorithms
operation_algorithm_mapping = {
    "Search by ID": {
        "Algorithm": "Hash Map",
        "Justification": "Provides O(1) average case time complexity for lookups, ideal for large data
    },
    "Search by Name": {
        "Algorithm": "Hash Map",
        "Justification": "Similar to ID search, it allows for efficient lookups based on product names
    },
    "Sort by Price": {
        "Algorithm": "Timsort",
        "Justification": "Timsort is a hybrid sorting algorithm derived from merge sort and insertion
    },
    "Sort by Stock Quantity": {
        "Algorithm": "Timsort",
        "Justification": "Same as sorting by price, Timsort is efficient for sorting large datasets wi
    }
}
# Print the operation to algorithm mapping
print("\nOperation to Algorithm Mapping:")
for operation, details in operation_algorithm_mapping.items():
    print(f"{operation}: {details['Algorithm']} - {details['Justification']}")
```

Output:



Justification:

An inventory system requires fast searching and sorting due to large data size and frequent updates. Hash Maps are used for searching because they provide instant access to products. Quick Sort and Merge Sort are chosen for efficient sorting by price and quantity. This task shows how algorithm selection depends on performance requirements and real-time usage.

Task 5: Real-Time Stock Data Sorting & Searching

Scenario:

An AI-powered FinTech Lab at SR University is building a tool for analyzing stock price movements. The requirement is to quickly sort stocks by daily gain/loss and search for specific stock symbols efficiently.

• Use GitHub Copilot to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Price).
• Implement sorting algorithms to rank stocks by percentage change.
• Implement a search function that retrieves stock data instantly when a stock symbol is entered.
• Optimize sorting with Heap Sort and searching with Hash Maps.
• Compare performance with standard library functions (sorted(), dict lookups) and analyze trade-offs.

Prompt:

Build a **real-time stock analysis tool** in Python for an AI-powered FinTech Lab. Use **GitHub Copilot / AI assistance** to fetch or simulate stock data containing **Stock Symbol, Opening Price, and Closing Price**.

Implement a sorting algorithm to **rank stocks by daily percentage gain/loss**, optimizing the solution using **Heap Sort**.
Implement an efficient **search function using Hash Maps** to instantly retrieve stock details when a stock symbol is entered.
Compare the performance of custom algorithms with **Python standard library functions (sorted(), dictionary lookups)** and analyze the **trade-offs**.

Code:

C: > Users > Hello > 12.5.py > ...

```python
#Build a real-time stock analysis tool in Python for an AI-powered FinTech Lab.Use GitH
#Implement a sorting algorithm to rank stocks by daily percentage gain/loss, optimizing
#Implement an efficient search function using Hash Maps to instantly retrieve stock det
#Compare the performance of custom algorithms with Python standard library functions (s
import random
import time
import heapq
class Stock:
    def __init__(self, symbol, opening_price, closing_price):
        self.symbol = symbol
        self.opening_price = opening_price
        self.closing_price = closing_price

    def percentage_gain_loss(self):
        return ((self.closing_price - self.opening_price) / self.opening_price) * 100
def generate_stock_data(num_stocks):
    stock_data = []
    for i in range(num_stocks):
        symbol = f'STK{i+1}'
        opening_price = round(random.uniform(100, 500), 2)
        closing_price = round(random.uniform(100, 500), 2)
        stock_data.append(Stock(symbol, opening_price, closing_price))
    return stock_data
def heap_sort_stocks(stocks):
    heap = []
    for stock in stocks:
        heapq.heappush(heap, (-stock.percentage_gain_loss(), stock))
    sorted_stocks = [heapq.heappop(heap)[1] for _ in range(len(heap))]
    return sorted_stocks
```

```
24  def heap_sort_stocks(stocks):
29      return sorted_stocks
30  def search_stock(stocks, symbol):
31      stock_map = {stock.symbol: stock for stock in stocks}
32      return stock_map.get(symbol, None)
33  def main():
34      num_stocks = 1000
35      stock_data = generate_stock_data(num_stocks)
36
37      start_time = time.time()
38      sorted_stocks = heap_sort_stocks(stock_data)
39      heap_sort_time = time.time() - start_time
40
41      start_time = time.time()
42      sorted_stocks_builtin = sorted(stock_data, key=lambda x: x.percentage_gain_loss(),
43      builtin_sort_time = time.time() - start_time
44
45      print(f"Heap Sort Time: {heap_sort_time:.6f} seconds")
46      print(f"Built-in Sort Time: {builtin_sort_time:.6f} seconds")
47
48      # Test search function
49      test_symbol = 'STK500'
50      stock_details = search_stock(stock_data, test_symbol)
51      if stock_details:
52          print(f"Stock found: {stock_details.symbol}, Opening Price: {stock_details.open
53      else:
54          print(f"Stock with symbol {test_symbol} not found.")
55  if __name__ == "__main__":    main()
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                    Python

PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Hello\AppData\Local\Pytho
.exe c:/Users/Hello/12.5.py
Heap Sort Time: 0.001910 seconds
Built-in Sort Time: 0.000265 seconds
Stock found: STK500, Opening Price: 105.89, Closing Price: 398.61
PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code>
```

Justification:

Stock market analysis requires real-time performance and fast data access. Heap Sort is used to rank stocks by percentage gain/loss efficiently, while Hash Maps enable instant stock symbol search. Comparing custom algorithms with Python built-in functions helps understand trade-offs between learning concepts and practical efficiency in real-world financial applications.